

Kayak Travel Simulation Platform

Distributed Systems Final Project

A scalable, AI-powered, microservices-based travel engine demonstrating advanced distributed systems principles including event-driven architecture, semantic caching, and multi-agent orchestration.

Team 11: Akshit Tyagi, Arya Mehta, Devanshee Vyas, Jane Heng, Jay Hemnani, Zoheb Waghu



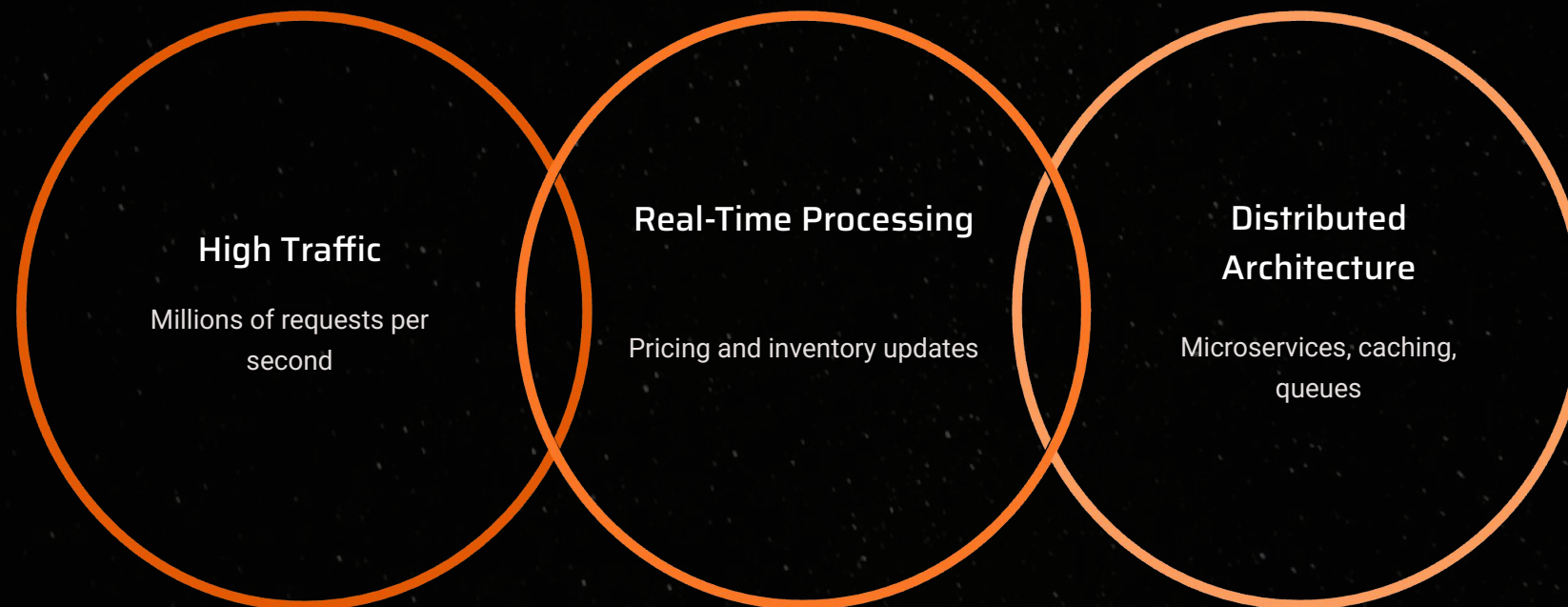
Problem Statement: Why We Built This System

Scale & Performance Challenges

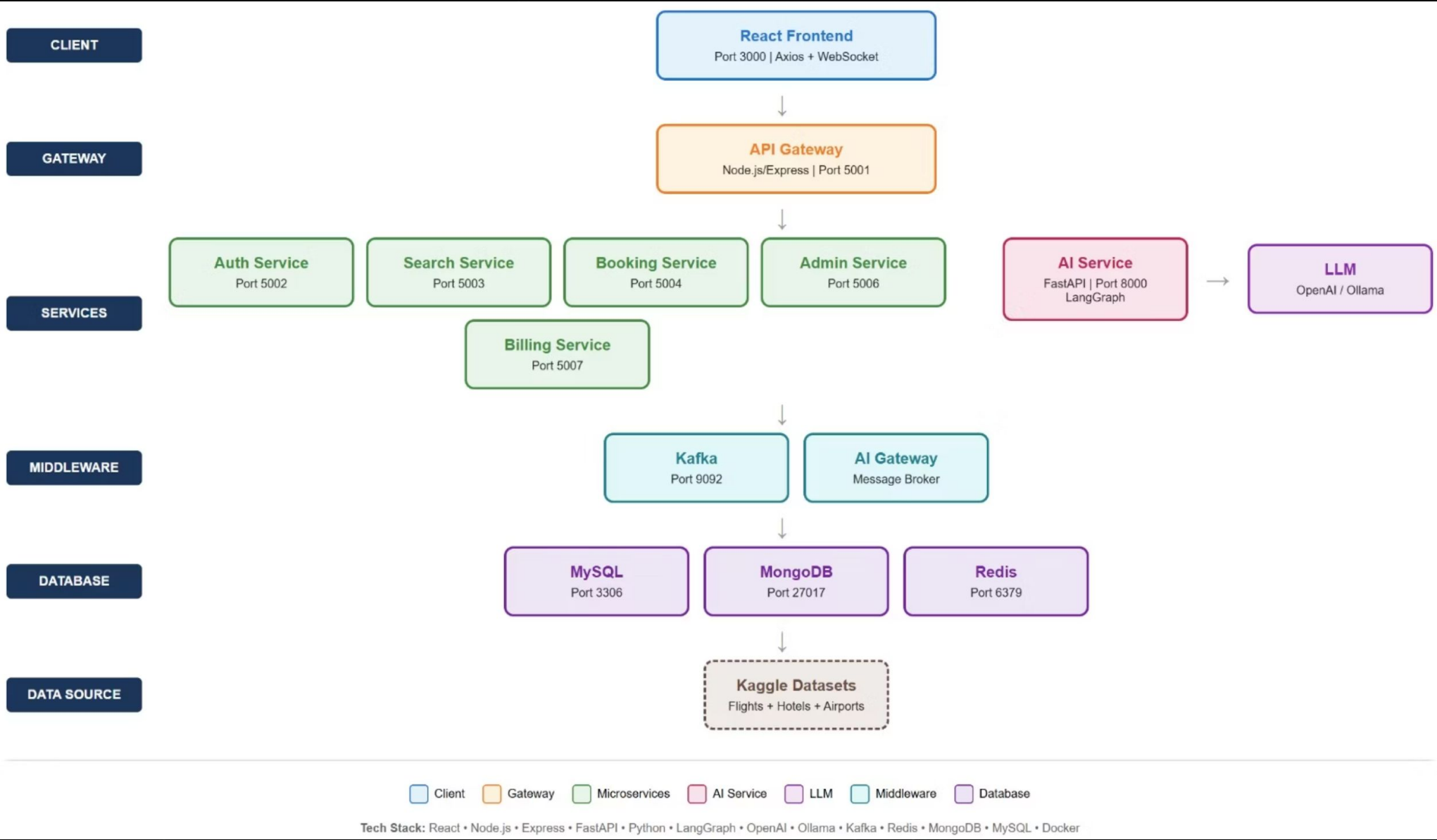
Modern travel platforms must handle millions of concurrent searches while maintaining sub-second response times. Real-time pricing and inventory changes require sophisticated caching strategies and event processing. Traditional monolithic architectures fail to scale horizontally, creating bottlenecks during peak travel seasons when search volumes spike 300%+.

Distributed Systems Requirements

- Low-latency responses across geographic regions
- High availability with fault tolerance
- Real-time data synchronization
- Elastic scaling for variable load

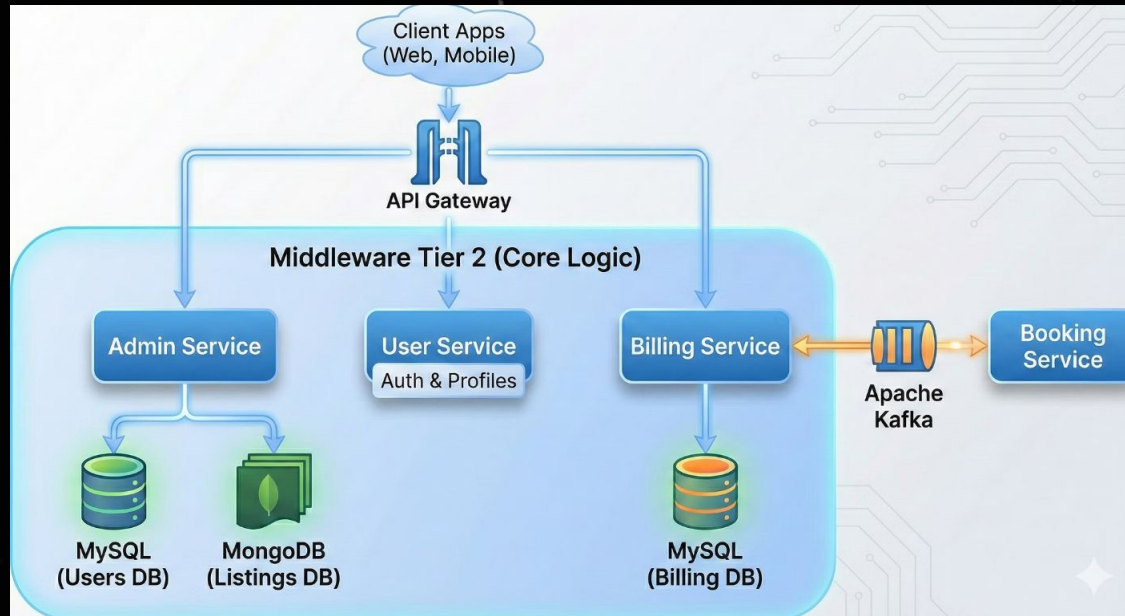


Full System Architecture Overview



Five-tier distributed architecture with React frontend, API gateway, microservices layer, Kafka event streaming, and polyglot database persistence.

Middleware Tier Architecture



Design Principle: Each service owns its domain logic and database, enabling independent deployment and horizontal scaling. Services communicate via REST APIs and asynchronous Kafka events.

Admin Service

Port 3001: Listings & Analytics

User Service

Port 3002: Auth & Profiles

Billing Service

Port 3003: Invoices & Payments

Booking Service

Port 3004: Reservations

Microservices Breakdown



User Service

Handles authentication with JWT tokens, user profile management, and preference storage. Integrates with Redis for session management and implements rate limiting per user tier.



Search Service

Queries MongoDB collections for flights and hotels with complex filters. Implements semantic caching using embeddings to reduce database load by 70% on repeated queries.



Booking Service

Manages transactional workflows with distributed locks, inventory reservations, and rollback mechanisms. Ensures atomicity across payment and confirmation steps.



Admin Service

Provides provider management, listing creation, and real-time analytics dashboards. Aggregates metrics from Kafka streams for revenue tracking and city-wise performance.



Billing Service

Generates invoices, processes payments, and maintains transaction history. Implements idempotency keys to prevent duplicate charges during network retries.

API Gateway: Central Orchestration

Gateway

Responsibilities

- Route requests from port 3000 to services 3001-3006
- JWT validation and user context injection
- Rate limiting with token bucket algorithm
- Request logging and distributed tracing

0

1 Request Ingress

Client sends request with auth headers

0

2 Authentication

Validate JWT and extract user ID

0

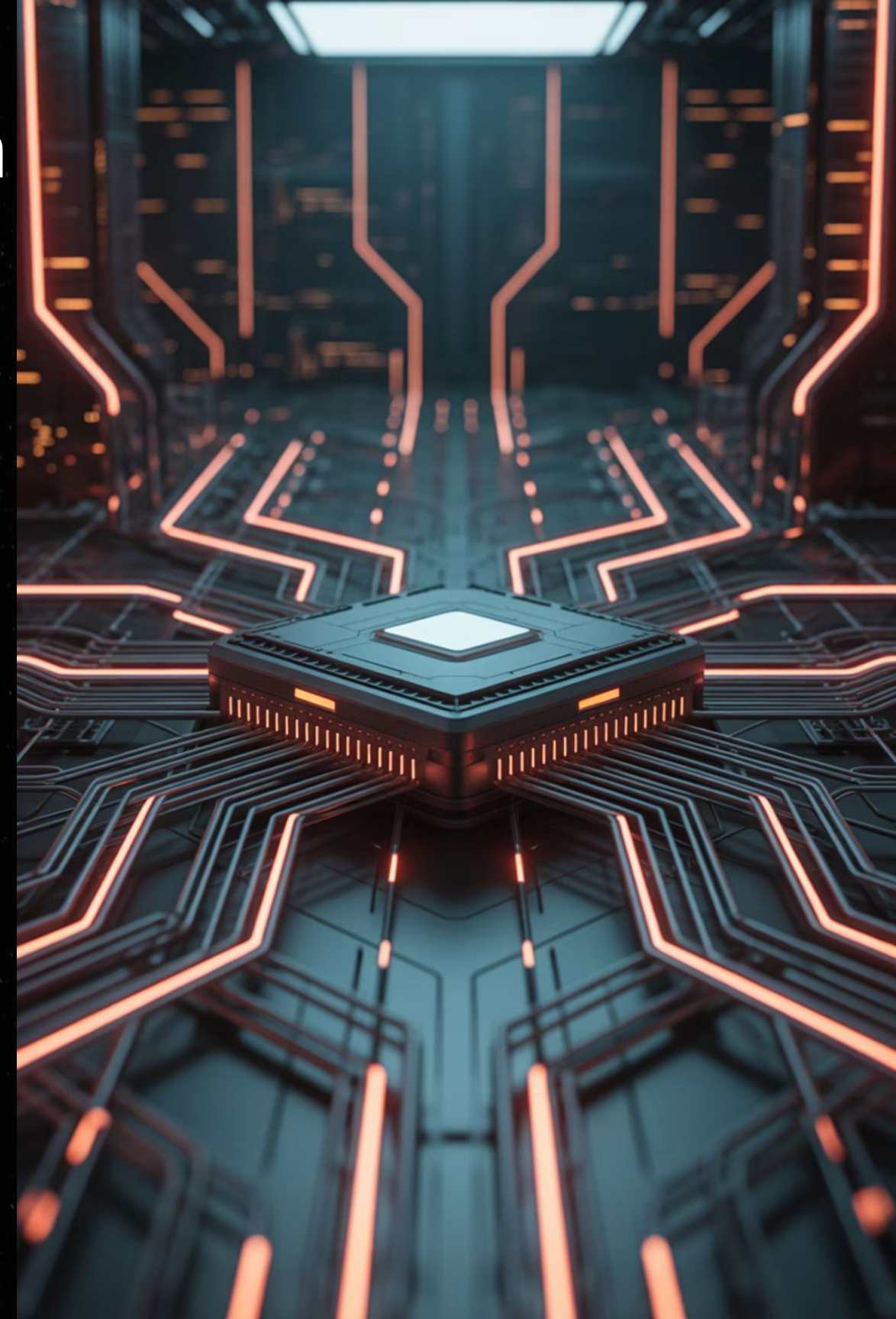
3 Service Resolution

Map route to target microservice port

0

4 Proxy & Response

Forward request and return service response





Database Strategy: Polyglot Persistence

MySQL for Transactional Data

Relational database optimized for ACID compliance and complex joins. Stores user accounts, bookings, billing records, and audit logs requiring strong consistency guarantees.

Features: Row-level locking, foreign key constraints, table partitioning by date ranges for efficient archival.

MongoDB for Search Analytics

Document database for flexible schemas and horizontal scaling.

Stores flight listings, hotel catalogs, user reviews, and high-volume event logs from Kafka streams.

Features: Compound indexes on search fields, aggregation pipelines for analytics, sharding by geographic region.

MySQL Schema Deep Dive



Users Table

Stores authentication credentials, profile data, and preferences.
Indexed on email and user_id for fast lookups during login.



Bookings Table

Transactional records with foreign keys to users and providers.
Partitioned by booking_date for efficient querying of recent reservations.



Billing Table

Invoice records linked to bookings.
Implements row-level locking during payment processing to prevent race conditions.

Optimization Strategies: Composite indexes on (user_id, booking_date), partitioning by month for bookings older than 1 year, and read replicas for analytics queries to offload primary database.

MongoDB Schema Deep Dive

Collections Overview

- **Flights Collection**

Origin, destination, departure time, price, airline details. Compound index on (origin, destination, date).

- **Hotels Collection**

Location, amenities, room types, pricing. Geospatial index for proximity searches.

- **Reviews Collection**

User feedback with ratings. Text indexes for keyword search in review content.

- **Deal Events Log**

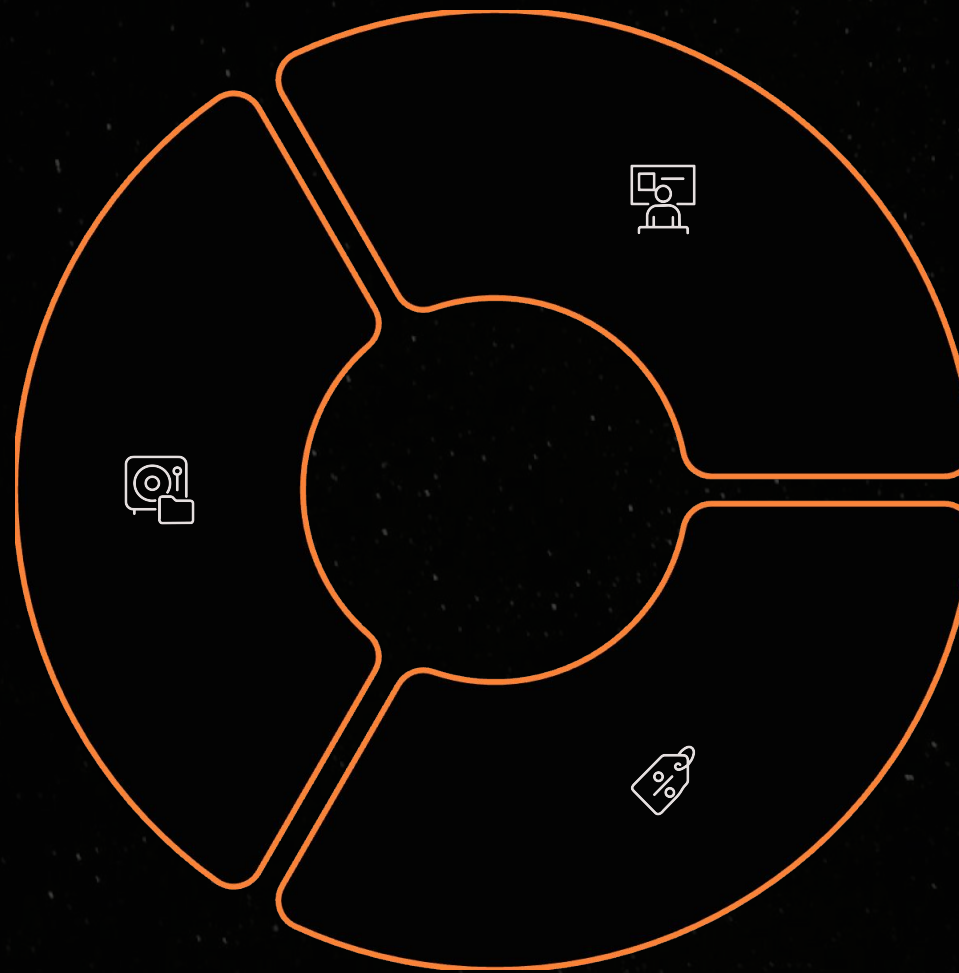
Time-series data from Kafka. TTL index for automatic deletion after 90 days.



Redis Caching Architecture

Semantic Cache

Stores search query embeddings and results. Matches similar queries using cosine similarity (threshold 0.85) to serve cached responses.



Session Store

Maintains user session data with JWT tokens. 30-minute TTL with sliding expiration on activity.

Deal Cache

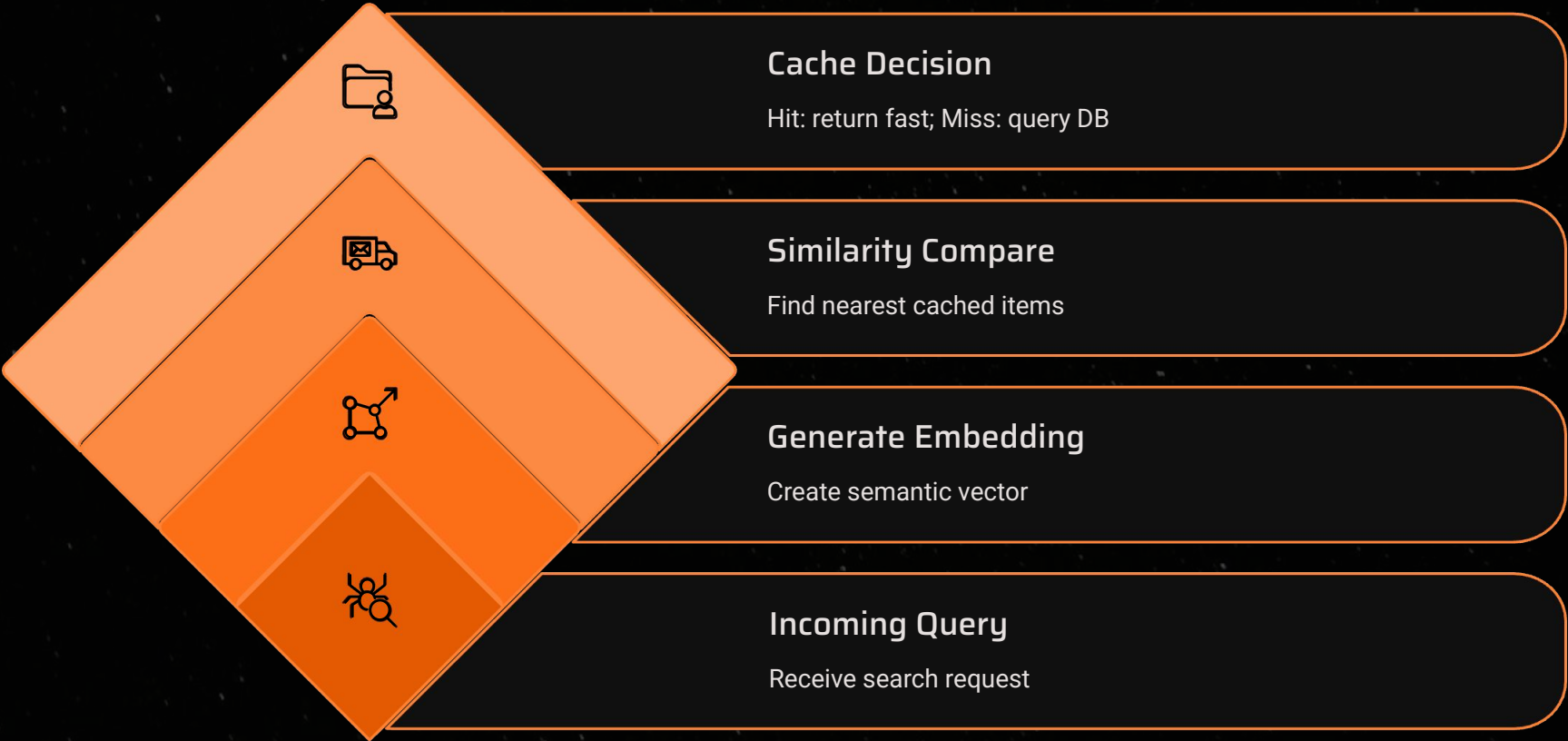
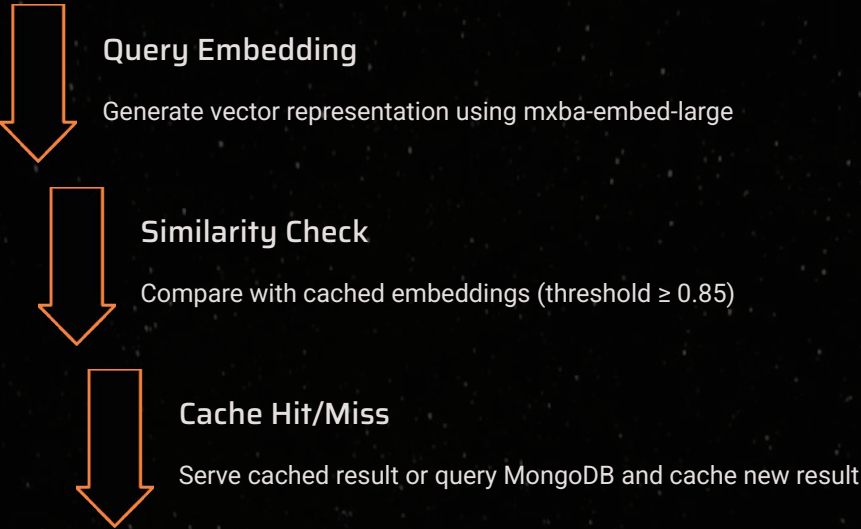
High-frequency deal data from Kafka. updated in real-time via pub/sub.

Search Caching: Hit/Miss Flow

Performance Impact

Before Caching: Average query latency of 800ms with direct MongoDB queries. Database CPU utilization at 75% during peak hours.

After Semantic Caching: Average response time (cached): 90ms. Nearly 8.89x faster with cache.



Embeddings & Semantic Similarity

Model: mxbai-embed-large

Generates 1024-dimensional vectors for natural language queries. Captures semantic meaning beyond keyword matching.

Similarity Threshold: 0.85 cosine similarity ensures high-quality cache hits while avoiding false positives.

Example: "cheap flights to Paris" matches "affordable plane tickets to France" with 0.89 similarity.

```
from ollama import Client

# Generate embeddings for semantic similarity
ollama_client = Client(host=settings.OLLAMA_BASE_URL)
embedding = ollama_client.embeddings(
    model=settings.OLLAMA_EMBEDDING_MODEL, # mxbai-embed-large
    prompt=query
)

# Semantic cache: find similar past queries
similarity = cosine_similarity(query_embedding, cached_embedding)
if similarity > SEMANTIC_SIMILARITY_THRESHOLD: # 0.85
    return cached_response
```

~40%

Cache Hit Rate

Queries served from semantic cache

90ms

Avg Response Time

With caching enabled

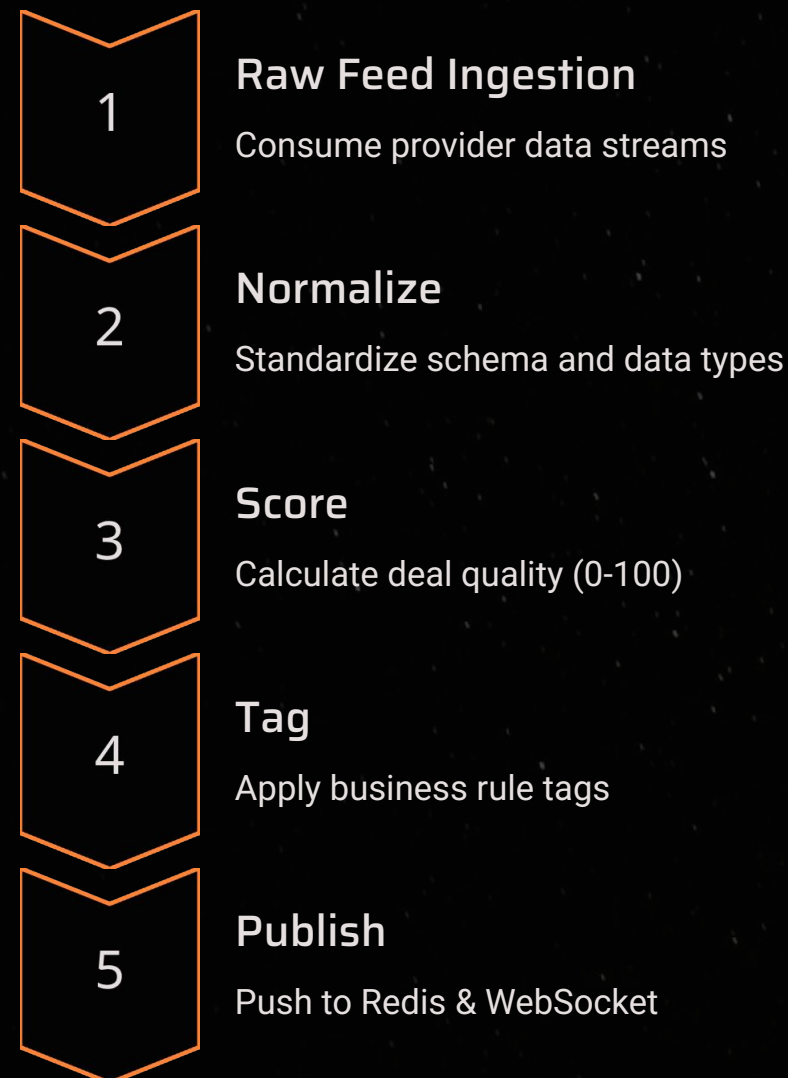
0.85

Similarity Threshold

Cosine similarity cutoff

Deals Agent: Kafka Deal Pipeline Architecture

Event-driven pipeline processing raw deal feeds through normalization, scoring, and tagging stages. Each stage is a separate Kafka consumer-producer maintaining exactly-once semantics.



Deal Tagging Logic

Promo Tag

Applied when discount exceeds 15% or explicit promotional code is present in deal metadata.

Scarce Tag

Triggered when remaining inventory falls below 10 units or booking velocity exceeds 5 reservations per hour.

Refundable Tag

Set for bookings allowing full or partial cancellation within 24-48 hours of departure.

Amenity Tags

Pet-friendly, pool access, free WiFi, parking included. Parsed from provider amenity lists.

[illegible]

Deal Scoring Algorithm (0-100)

Factor	Max Points	Rule
Discount	30 pts	1 point per 1% below 30-day average (max 30%)
Scarcity	20 pts	< 5 seats/rooms = 20 pts, < 10 = 10 pts
Promo	15 pts	Active promotion = 15 pts
Direct Flight	10 pts	Non-stop flight = 10 pts
Star Rating	12 pts	Hotel stars × 2.4 (5-star = 12 pts)
Refundable	5 pts	Free cancellation = 5 pts
Amenities	8 pts	Breakfast (4) + Pool (2) + WiFi (2)

Weighted Scoring Model

- **Discount (30%)**: Price reduction vs. historical average
- **Scarcity (20%)**: Inventory urgency factor
- **Promo Status (15%)**: Active promotional campaign
- **Direct Flight (10%)**: No layovers bonus
- **Rating (15%)**: Provider review score
- **Refundable (5%)**: Cancellation flexibility
- **Amenities (5%)**: Additional perks count

Scores above 80 trigger real-time alerts. Recalculated every 5 minutes using sliding window aggregations.

Concierge Agent : Natural Language Interface

An LLM-powered conversational agent that understands user needs and provides personalized recommendations (support both OpenAI and Ollama):

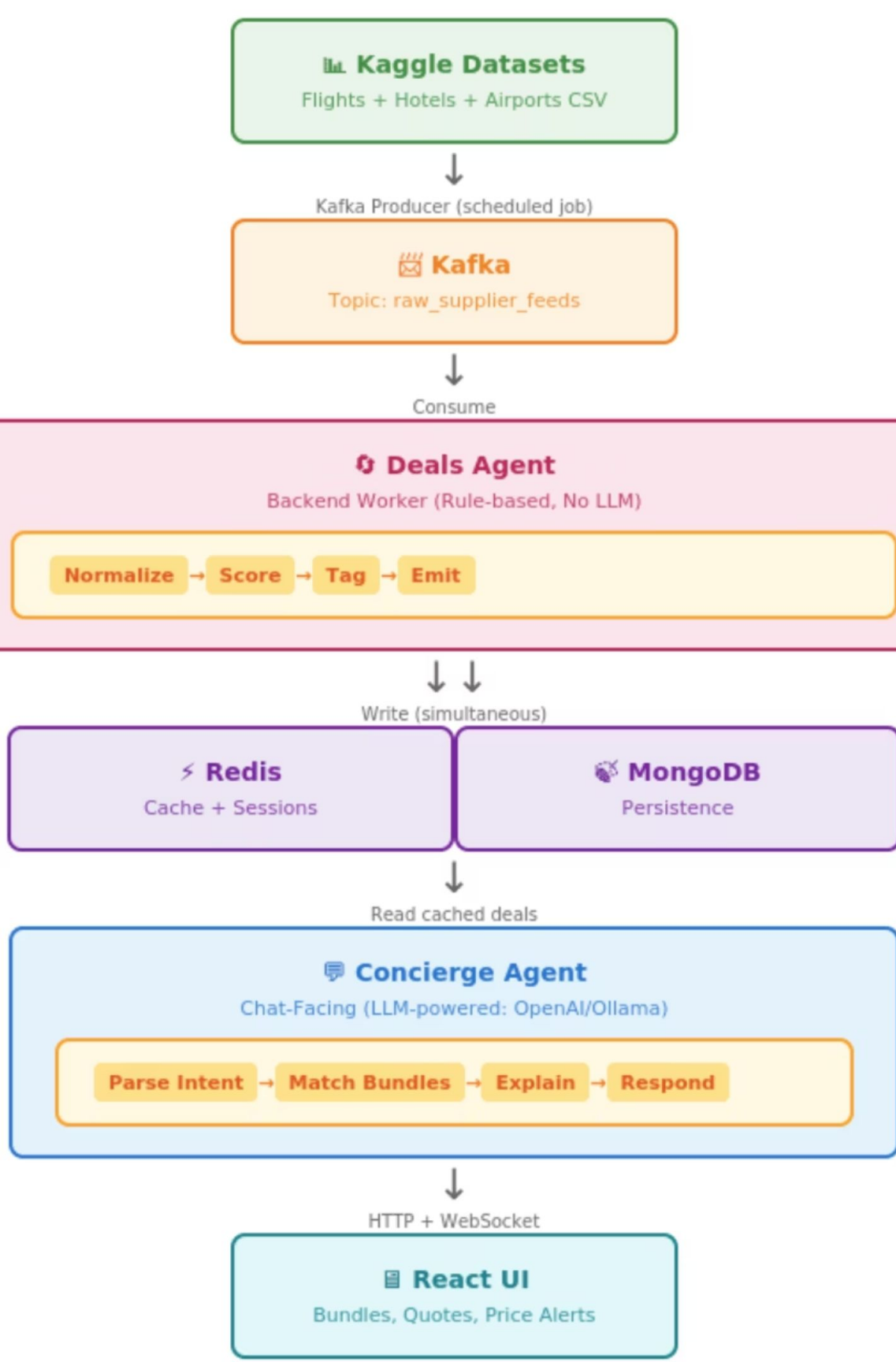
"Find me a weekend trip to a beach destination under \$800 with direct flights and pet-friendly hotels"

- **Perceive:** Receives natural language queries from users (dates, budget, preferences)
- **Decide:** Uses LLM (OpenAI/Ollama) to parse intent, match bundles, and generate explanations
- **Act:** Returns flight+hotel bundles, creates price watches, answers policy questions, generates booking quotes

On-demand, responds to user requests via HTTP/WebSocket

Maintains conversation context via Redis session store

Composes bundles from cached deals with Fit Score ranking



MRKL Tools

AI agents decides when to use which tool to analyze user intent, evaluate policies, monitor pricing trends, and synthesize personalized recommendations.

Intent Parser

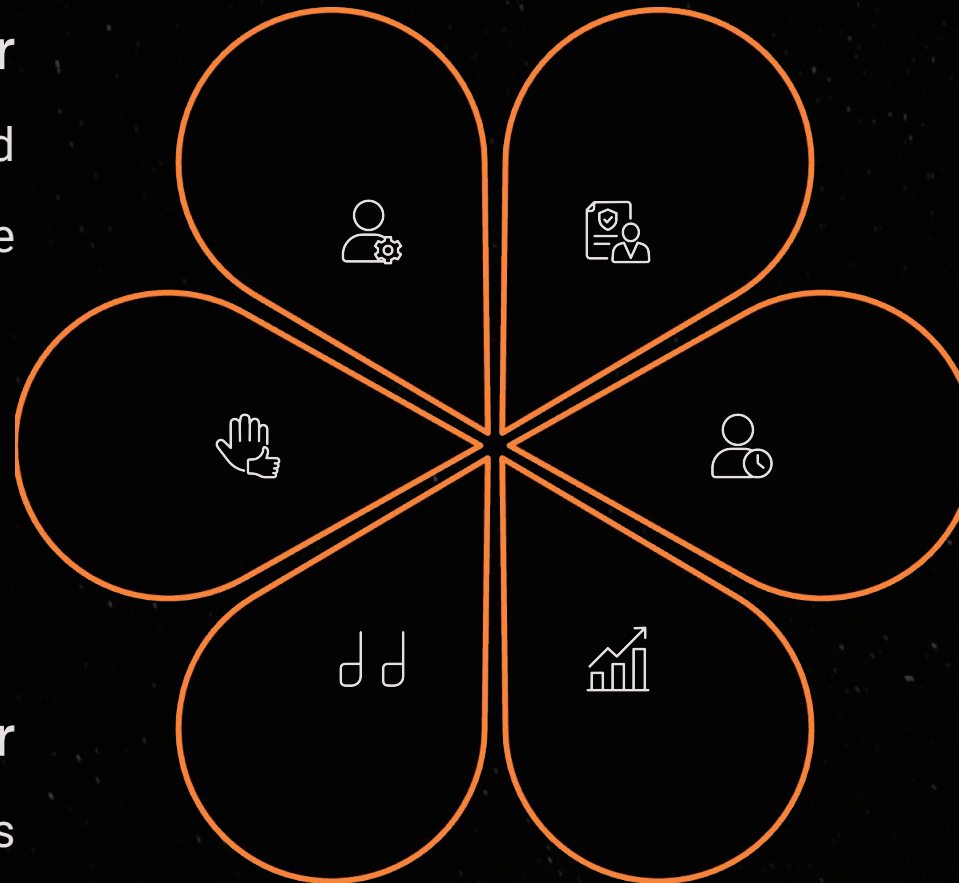
Extracts destination, dates, budget, and constraints from natural language

Bundle Matcher

Composes flight+hotel bundles from cached deals, ranks by Fit Score

Quote Generator

Generates fare estimates



Policy Agent

Validates business rules

Watch Creator

Creates price/inventory threshold alerts, triggers WebSocket push

Price Analyzer

Compares current price to 30-day average, returns verdict (GREAT DEAL / FAIR / ABOVE AVG)

WebSocket Real-Time Alerts

```
const ws = new WebSocket('ws://localhost:8000/api/ai/events?user_id=user123');  
ws.onmessage = (event) => {  
  const data = JSON.parse(event.data);  
  // Handle: price_alert, inventory_alert, deal_found, watch_triggered  
};
```

Maintains persistent connections with automatic reconnection. Messages serialized as JSON with event type discriminator.

price_alert

Sent when tracked flight/hotel price drops below user threshold. Includes old price, new price, and percentage change.

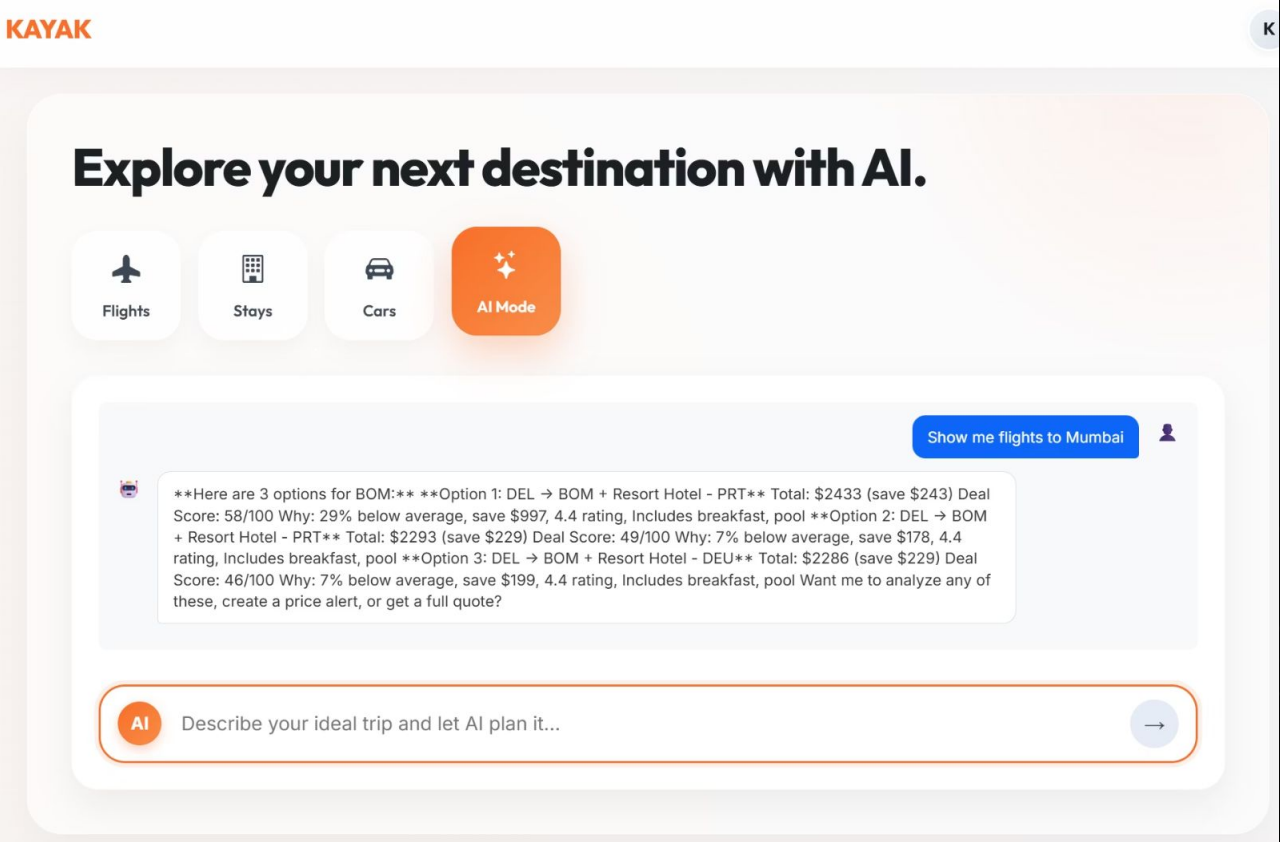
inventory_alert

Triggered when low inventory detected (< 5 rooms/seats). Encourages immediate booking action.

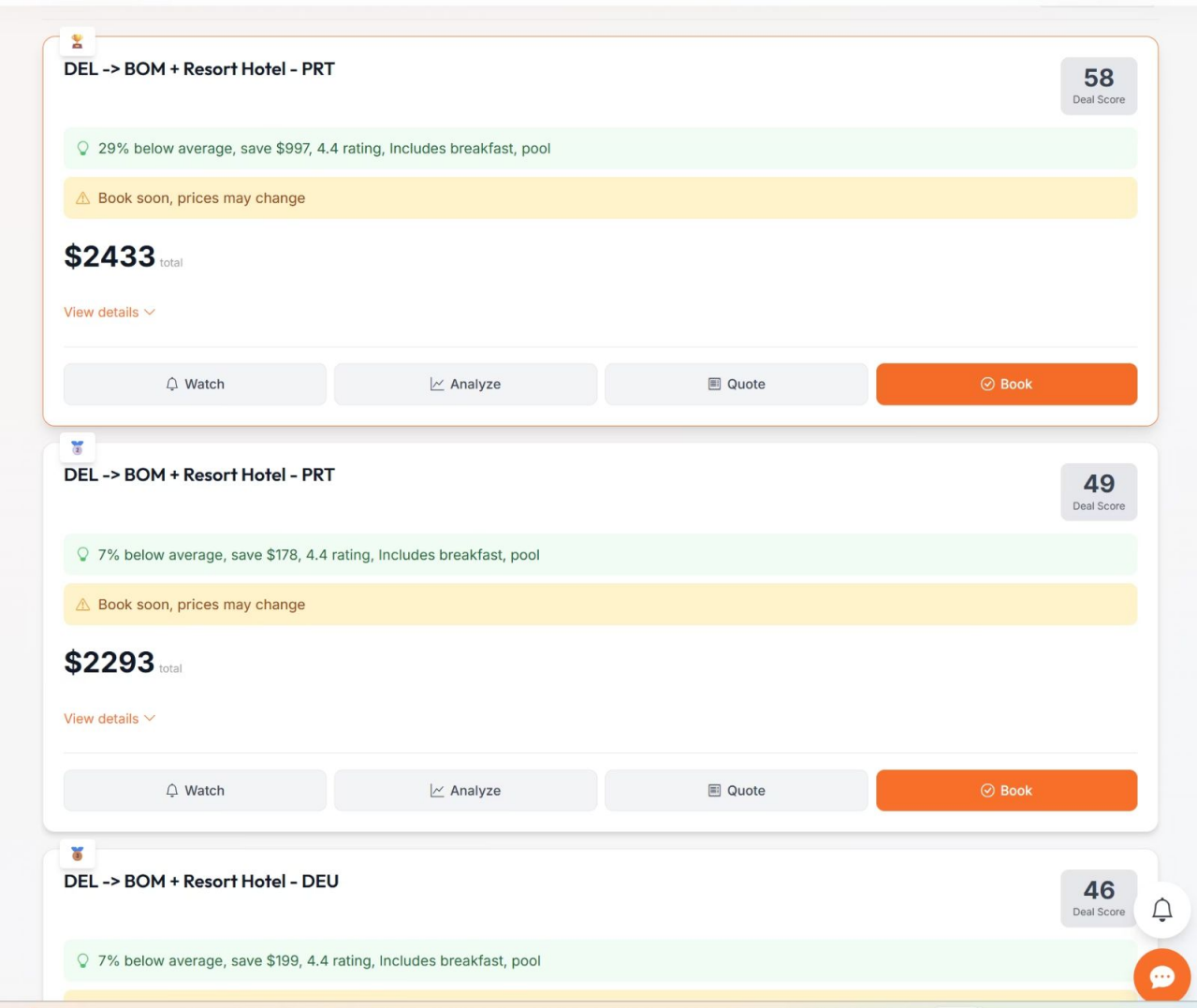
deal_found

High-score deals matching user preferences. Pushed in real-time from Kafka consumers.

Bundle Recommendations Interface



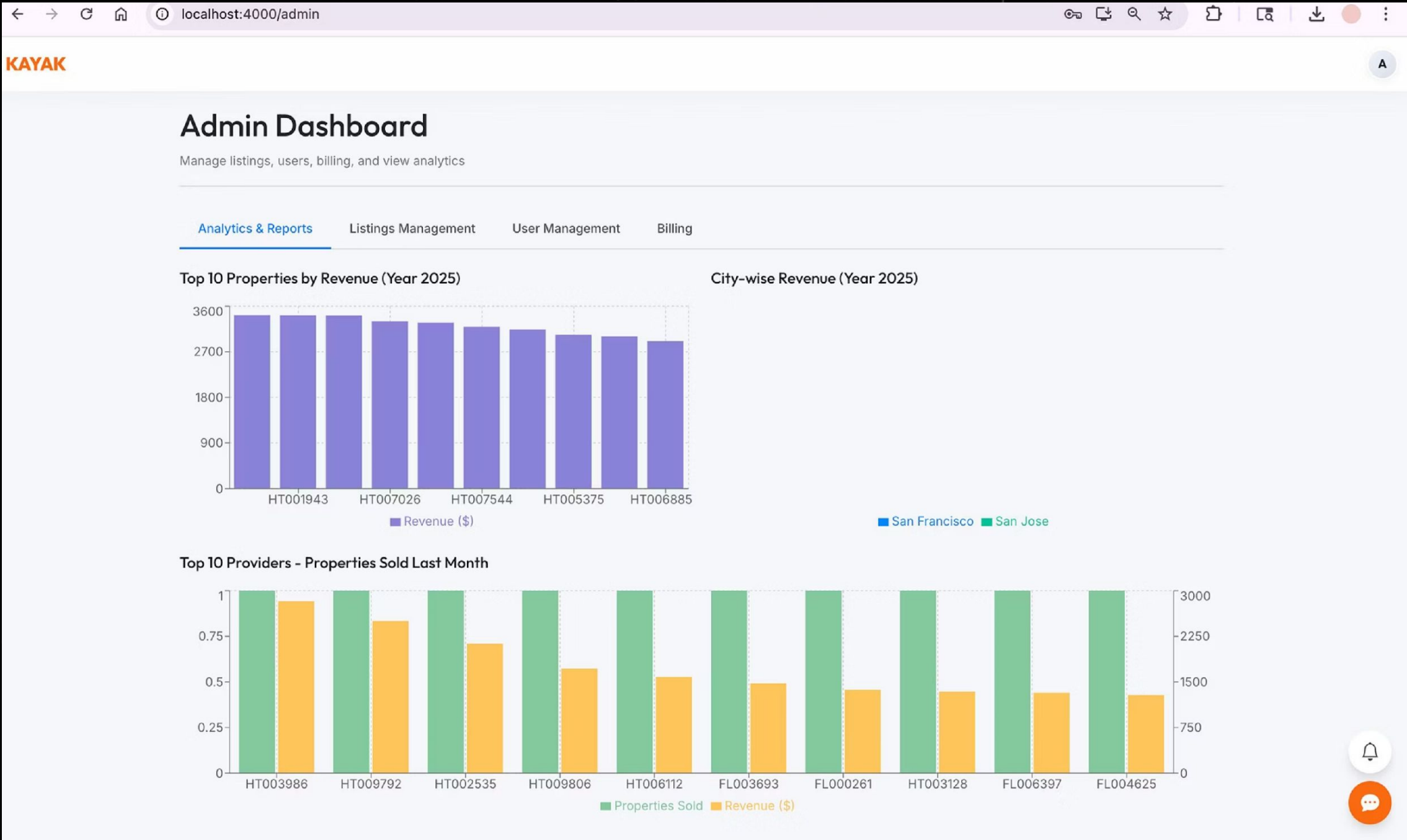
Natural language query input



AI-powered bundle recommendations

AI agent analyzes flight-hotel combinations, calculating total scores based on price competitiveness, convenience (proximity, timing), and user preferences. Each bundle displays combined score (0-100), cost breakdown, and key highlights.

Admin Dashboard: Real-Time Analytics





Performance Analysis Overview

Test Configuration

Our load testing framework simulated real-world traffic patterns to identify bottlenecks and validate optimization strategies across critical user paths.

- **100 concurrent users** — sustained load profile
- **Apache JMeter** — industry-standard load generator
- **Two primary flows** — search and booking transactions
- **5-minute warm-up** — eliminated cold-start anomalies

Architecture Under

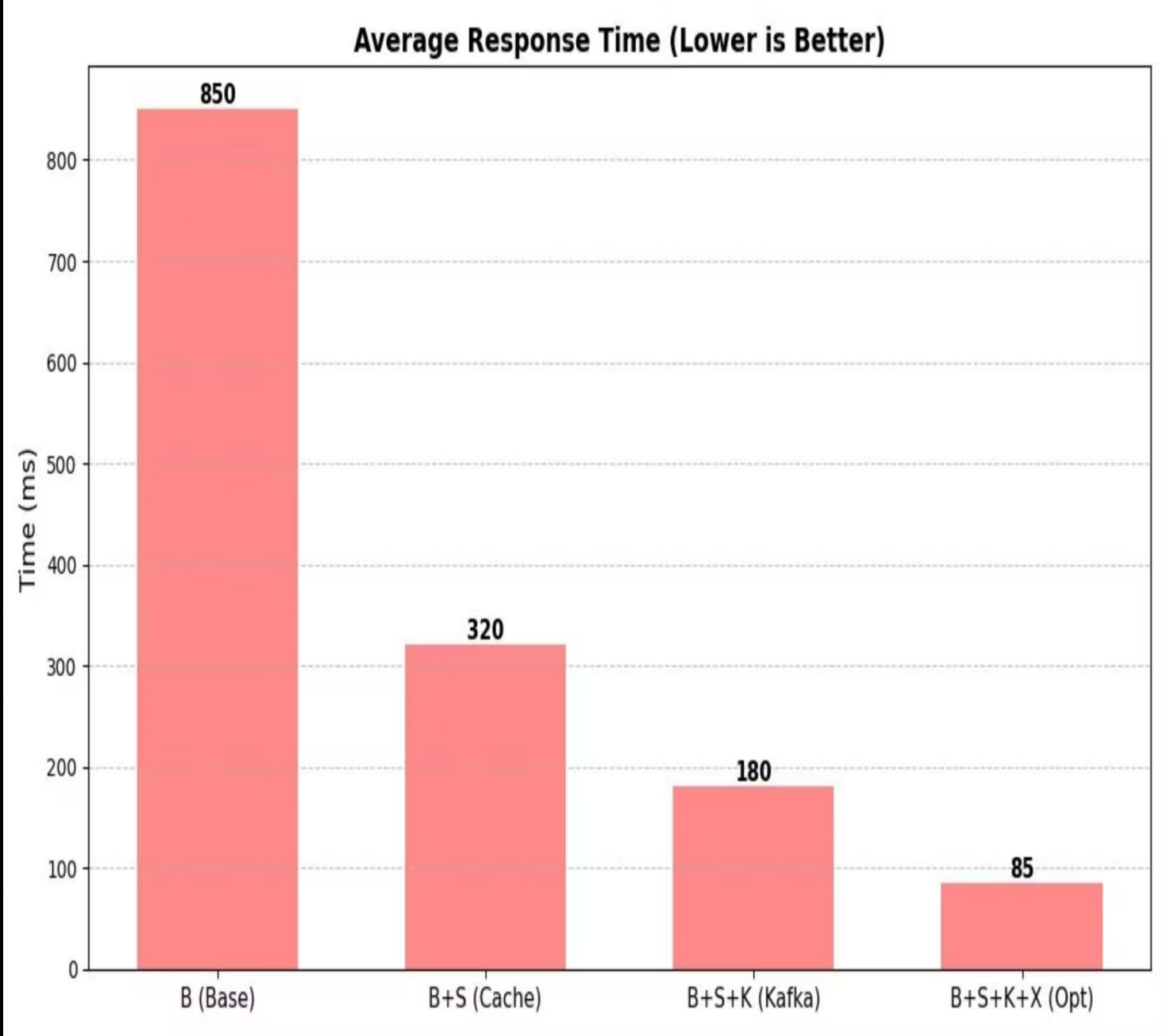
Test

We evaluated four distinct system configurations, measuring incremental improvements as each optimization layer was introduced.

1. Baseline: Monolithic architecture with direct DB calls
2. Redis cache layer for hot data paths
3. Kafka-based async processing pipeline
4. Fully optimized: combined strategies + query tuning

Average Response Time Analysis

Lower latency = better user experience



Baseline: 850ms

Direct database queries created severe contention. Every request triggered multiple synchronous DB round-trips, saturating connection pools during peak load.

Cache Layer: 320ms

Redis cache reduced DB load by 70%, eliminating repeated queries for frequently accessed data. Cache hit rate: 82% for search operations.

Kafka Queue: 180ms

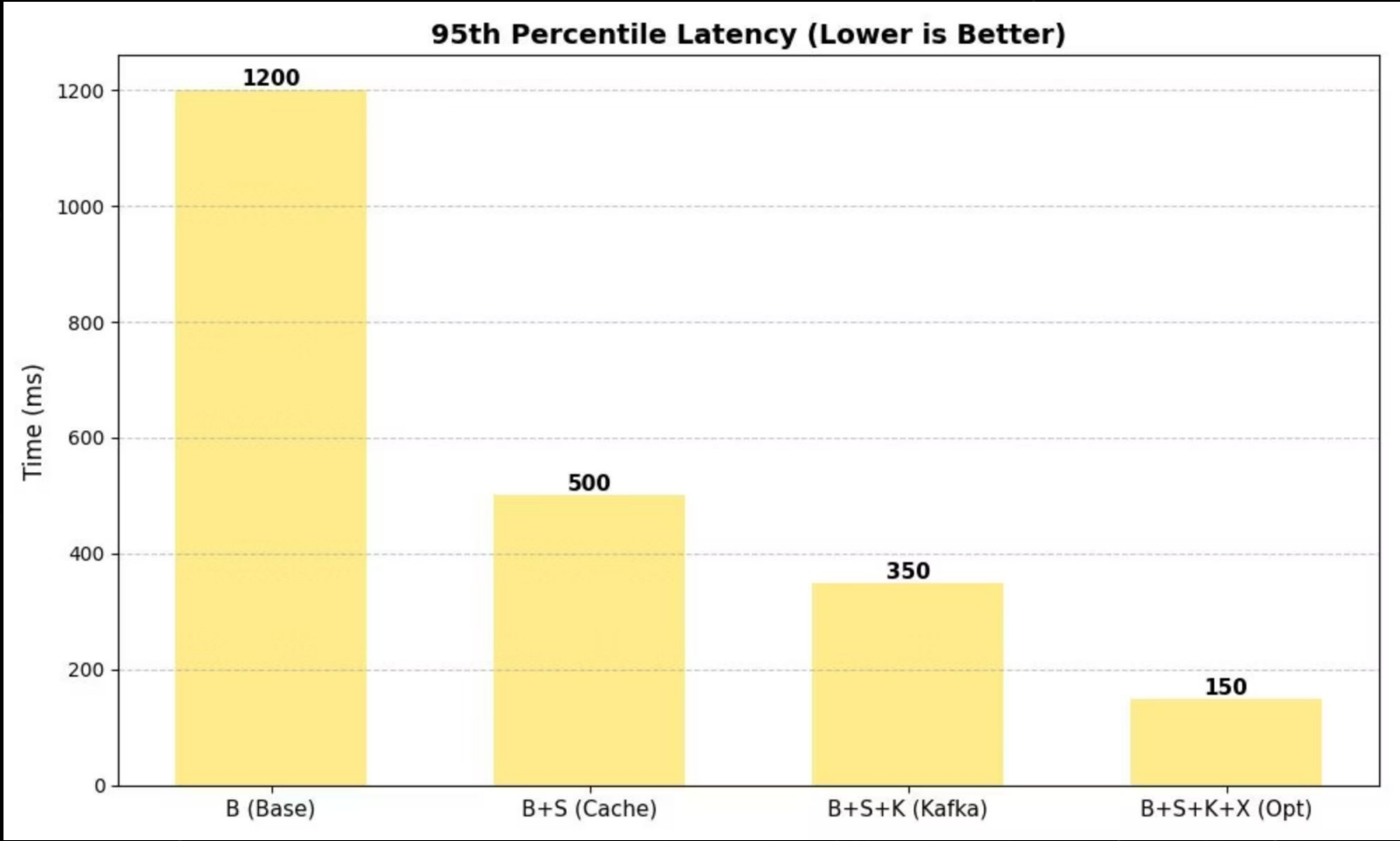
Async event processing offloaded non-critical writes. User-facing requests returned immediately while background workers handled persistence.

Optimized: 85ms

Query optimization, connection pooling, and index tuning delivered 10× improvement. Sub-100ms response time meets SLA targets.

Tail Latency Deep Dive

95th percentile reveals worst-case user experience



- 1

Baseline: 1,200ms
Lock contention and thread pool exhaustion caused catastrophic degradation for 5% of requests.
- 2

Cache: 500ms
Reduced database pressure improved tail behavior, but cache misses still triggered slow paths.
- 3

Kafka: 350ms
Async processing eliminated write blocking, stabilizing P95 latency under sustained load.
- 4

Optimized: 150ms
Consistent sub-200ms P95 latency ensures reliable experience even during traffic spikes.

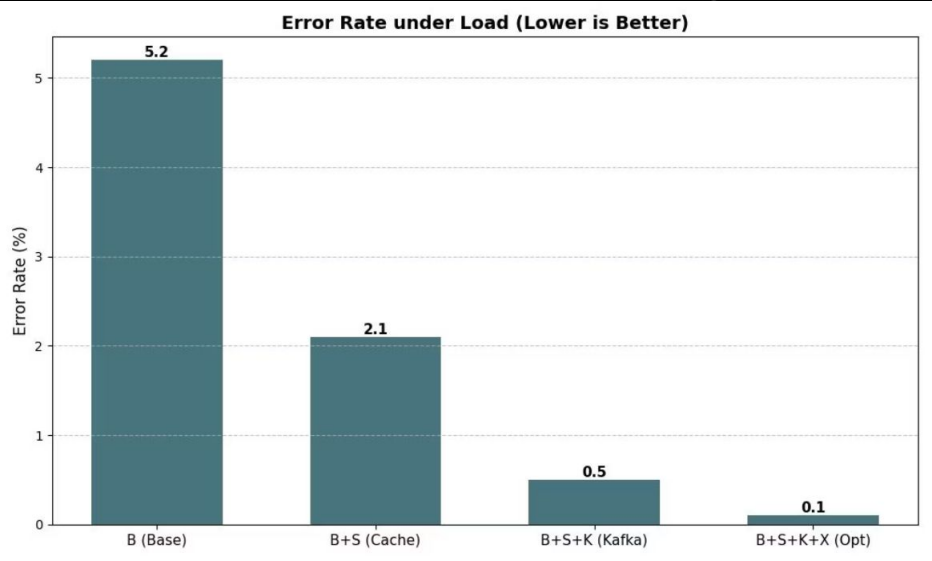
Why P95 Matters

Average latency masks outliers. The 95th percentile reveals the experience for your most engaged users—those making multiple requests per session.

High tail latency often indicates resource exhaustion, cascading failures, or architectural bottlenecks that only manifest under load.

Error Rate & Throughput Correlation

Error Rate Reduction



5.2%

Baseline Error Rate

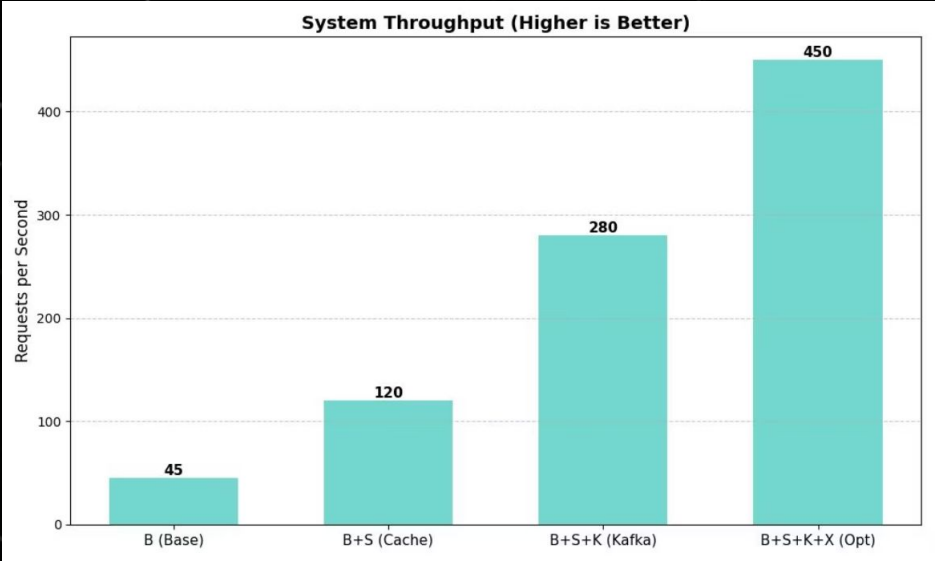
Connection pool exhaustion caused cascading timeouts. Database rejecting 1 in 20 requests during peak traffic.

0.1%

Optimized Error Rate

52× reduction in failures. Errors now limited to legitimate edge cases and transient network issues.

Throughput Scaling



45

Baseline RPS

System saturated at 45 requests/second. Further load caused exponential latency increase and failures.

450

Optimized RPS

10× throughput improvement with linear scaling characteristics. Headroom for 3× additional growth.

The correlation is clear: **reduced error rates directly enabled higher throughput**. By eliminating resource contention and timeouts, the system could handle sustained load without degradation.

System Limitations & Future Work

Current Limitations

Single-Region Deployment

No geographic redundancy. Network latency impacts international users. Single point of failure for regional outages.

Single-Node Redis

Cache layer lacks HA. Node failure forces cold restart with full DB load. No automatic failover mechanism.

Kafka Broker Dependency

Single broker creates bottleneck. Message loss risk during restarts. Limited partition scalability.

LLM Inference

AI-powered features add 200-400ms overhead. Model inference runs synchronously. No request batching optimization.

Roadmap Priorities

0

1

Multi-Region Active-Active
Deploy across AWS us-east-1, eu-west-1, ap-southeast-1. Route53 geo-routing with DynamoDB global tables.

0

2

Redis Cluster Mode
Migrate to Redis Cluster with automatic sharding and failover. Target: 99.99% cache availability.

0

3

Circuit Breaker Pattern
Implement Hystrix-style fault isolation. Graceful degradation when dependencies fail. Fast-fail to prevent cascade.

0

4

Distributed Tracing
Deploy Jaeger + OpenTelemetry for end-to-end observability. Correlate latency across service boundaries.



Questions?



System design & architecture decisions



Metrics, benchmarks & optimization



Fault tolerance & scaling strategies